
CONTRALORÍA GENERAL DE LA NACIÓN

Sistema de Inteligencia Artificial para Auditores

FASE DE DESARROLLO – SIAIA-CGN v1.0

Construcción del Software • Digitalización del Código • Integración de Aplicativos

Versión	1.0
Fecha de emisión	Mayo 2026
Fase del proyecto	F2 – F3 (Meses 2 al 5)
Responsable	Líder de Desarrollo – Empresa Contratista
Estado	En Desarrollo
Clasificación	Confidencial

1. INTRODUCCIÓN Y PROPÓSITO DEL DOCUMENTO

1.1 Propósito

El presente documento describe de manera integral la Fase de Desarrollo del Sistema de Inteligencia Artificial para Auditores de la Contraloría General de la Nación (SIAIA-CGN v1.0). Abarca la construcción del software, la digitalización del código fuente, la implementación de los módulos funcionales y la integración de los aplicativos que conforman la plataforma.

Este documento rige las fases F2 y F3 del cronograma aprobado, correspondientes a los meses 2 al 5 del proyecto, y sirve como guía técnica vinculante para el equipo de desarrollo de la empresa contratista y el equipo de TI de la CGN.

1.2 Alcance de la Fase

La Fase de Desarrollo comprende las siguientes grandes actividades:

- Construcción y configuración del entorno de desarrollo seguro.
- Implementación del motor de Inteligencia Artificial (modelo de detección de anomalías).
- Desarrollo de los módulos de la aplicación web: ingesta, dashboard, reportes, alertas y repositorio.
- Digitalización y control de versiones del código fuente en repositorio institucional.
- Integración de los aplicativos internos y externos (SIIF Nación, Active Directory, SMTP).
- Pruebas unitarias y de integración durante el desarrollo (shift-left testing).

1.3 Documentos de Referencia

Este documento se fundamenta en los siguientes artefactos aprobados del proyecto:

- Ingeniería de Requerimientos SIAIA-CGN v1.0 (SRS - IEEE 830 adaptado) – Mayo 2026
 - Documento de Visión y Alcance – Sección 1 del SRS
 - Matriz de Trazabilidad de Requerimientos (RTM) – Sección 4 del SRS
 - Arquitectura Técnica del Sistema – Entregable F1
 - Resolución Orgánica CGN No. 7350 de 2013 (Manual de Auditoría)
 - Ley 1581 de 2012 (Protección de Datos) y Ley 1712 de 2014 (Transparencia)
-

2. ARQUITECTURA TÉCNICA DE DESARROLLO

2.1 Stack Tecnológico

El sistema SIAIA-CGN se construye sobre una arquitectura de microservicios desacoplados, desplegados en infraestructura soberana colombiana, conforme a la restricción RNF-09 y la Ley 1581 de 2012. A continuación se detalla el stack tecnológico aprobado:

Capa	Tecnología	Justificación técnica
Frontend	React.js 18 + TypeScript	SPA responsiva, alto rendimiento, amplio soporte de componentes para dashboards analíticos.
Backend API	Python 3.11 + FastAPI	Alto rendimiento asíncrono, nativo para librerías de IA/ML, tipado con Pydantic.
Motor de IA / ML	Python + scikit-learn + HuggingFace Transformers + spaCy	Ecosistema consolidado para NLP, series temporales y detección de anomalías.
Base de Datos Relacional	PostgreSQL 15	Robustez, extensiones para JSON/XML, cumplimiento ACID para datos financieros.
Base de Datos Vectorial	pgvector (extensión PostgreSQL)	Almacenamiento de embeddings para búsqueda semántica de hallazgos históricos.
Almacenamiento de Objetos	MinIO (S3-compatible)	Repositorio de documentos (PDF, XLSX, XML) en infraestructura on-premise.
Procesamiento de Colas	Apache Kafka	Ingesta masiva de documentos y eventos asíncronos de alta disponibilidad.
Autenticación	LDAP / Active Directory + OAuth 2.0 + JWT	Integración directa con directorio institucional CGN.
Orquestación	Docker + Kubernetes (K8s)	Despliegue contenedorizado, escalabilidad horizontal, alta disponibilidad.
CI/CD	GitLab CI / Jenkins	Automatización de pipelines de construcción, pruebas y despliegue.
Monitoreo	Prometheus + Grafana + ELK Stack	Observabilidad completa: métricas, logs y alertas de infraestructura.

2.2 Principios de Arquitectura

El desarrollo se rige por los siguientes principios arquitectónicos no negociables:

- Soberanía de datos: Ningún dato de auditoría sale de los servidores de la CGN en Colombia.
-

-
- Seguridad por diseño (Security by Design): Cifrado TLS 1.3 en tránsito y AES-256 en reposo desde la capa de código.
 - Separación de responsabilidades: Cada microservicio tiene un único dominio funcional.
 - Trazabilidad inmutable: Todo acceso y acción de usuario genera un log inmutable (RNF-04).
 - Escalabilidad horizontal: El sistema soporta mínimo 200 usuarios concurrentes (RNF-05).
 - Cobertura de pruebas: Mínimo 80% de cobertura en pruebas unitarias (RNF-07).
-

3. CONSTRUCCIÓN DEL SOFTWARE – MÓDULOS DEL SISTEMA

Esta sección describe la construcción de cada módulo funcional del SIAIA-CGN, su estructura interna, las tecnologías aplicadas y los criterios de aceptación técnica que deben cumplirse antes de pasar a la fase de pruebas.

3.1 Módulo de Ingesta y Procesamiento de Documentos (RF-01)

Objetivo: Permitir la carga masiva de documentos financieros y contractuales, extraer información estructurada y ponerla disponible para el motor de IA.

3.1.1 Componentes internos

- File Upload Service: API REST (POST /api/v1/documents/upload) que acepta multipart/form-data con validación de formato (PDF, XLSX, XML SIIF, CSV) y tamaño máximo de 50 MB por archivo.
- Document Preprocessor: Servicio en Python que aplica OCR (Tesseract/PaddleOCR) a PDFs escaneados, y parseo nativo para formatos estructurados (openpyxl para XLSX, lxml para XML SIIF).
- Data Extractor Pipeline: Pipeline de extracción con Apache Kafka que procesa lotes de 500 documentos de forma paralela mediante workers distribuidos.
- Metadata Indexer: Indexa en PostgreSQL las cifras, fechas, NIT de entidades, valores contractuales y tipo de documento extraídos.

3.1.2 Criterio de aceptación técnica

El módulo se acepta cuando procesa un lote de 500 documentos en menos de 10 minutos con una precisión de extracción de datos igual o superior al 95%, medida sobre un conjunto de validación anotado por auditores de la CGN.

3.2 Motor de Inteligencia Artificial – Detección de Anomalías (RF-02)

Objetivo: Analizar los datos financieros procesados y detectar patrones irregulares como sobrecostos, duplicidad de pagos, inconsistencias presupuestales, contratos sospechosos y desvíos de recursos.

3.2.1 Arquitectura del modelo

El motor de IA se compone de tres sub-modelos especializados que operan en pipeline:

Sub-modelo	Técnica aplicada	Tipo de irregularidad que detecta
Anomaly Detector	Isolation Forest + Autoencoder LSTM	Transacciones estadísticamente anómalas en series de tiempo presupuestales.
NLP Classifier	BERT fine-tuned (RoBERTa-base-spanish)	Inconsistencias en objetos contractuales, cláusulas sospechosas, proveedores relacionados.

Rule Engine	Motor de reglas basado en JSON Schema	Violaciones explícitas de normativa: pagos duplicados, valores por encima de topes legales, fechas inconsistentes.
--------------------	---------------------------------------	--

3.2.2 Proceso de entrenamiento

- Dataset de entrenamiento: Auditorías históricas de la CGN anonimizadas (mínimo 5 años de datos), proporcionadas según el supuesto SA-01 del SRS.
- División de datos: 70% entrenamiento, 15% validación, 15% prueba.
- Métricas objetivo: Precisión (Precision) $\geq 85\%$ y Recall $\geq 80\%$ sobre el dataset de prueba histórico.
- Versionamiento de modelos: Cada versión entrenada se almacena con MLflow para trazabilidad y rollback.
- Reentrenamiento: El modelo se reentrenará con nuevos datos cada trimestre en producción.

3.2.3 API del Motor de IA

El motor expone un endpoint interno: POST /api/v1/ai/analyze. Recibe el dataset financiero estructurado y devuelve una lista priorizada de hallazgos con nivel de confianza expresado en porcentaje. El tiempo máximo de respuesta por lote es de 60 segundos.

3.3 Módulo de Generación Automática de Reportes (RF-03)

Objetivo: Generar reportes de auditoría en formato .docx y .pdf bajo las plantillas institucionales vigentes de la CGN, incluyendo resumen ejecutivo, hallazgos detallados, evidencias y recomendaciones.

3.3.1 Componentes

- Template Engine: Motor de plantillas basado en Jinja2 + python-docx que inyecta hallazgos validados sobre la plantilla institucional CGN.
- PDF Renderer: Conversión automatizada docx $\rightarrow$ PDF mediante LibreOffice Headless, garantizando fidelidad visual.
- Digital Signature Service: Integración con firma digital institucional para el flujo de aprobación del Director (HU-05).
- Report Repository: Almacenamiento de reportes en MinIO con sello de tiempo y hash SHA-256 para verificación de integridad.

3.3.2 Criterio de aceptación técnica

El reporte se genera en menos de 2 minutos desde la aprobación del hallazgo. El formato cumple el 100% con la plantilla institucional vigente aprobada por la Dirección de Auditoría de la CGN.

3.4 Módulo de Dashboard – Panel de Control y Visualización (RF-04)

Objetivo: Proveer a los coordinadores y directores de auditoría un panel interactivo con indicadores clave, mapas de calor de riesgo fiscal, gráficos de tendencias y alertas en tiempo real.

- Frontend: Desarrollado en React.js con la librería Recharts/D3.js para visualizaciones complejas y Leaflet.js para mapas georreferenciados de entidades auditadas.
- Backend BFF (Backend for Frontend): Servicio intermedio en FastAPI que agrega datos de múltiples microservicios y los sirve optimizados para el frontend.
- WebSockets: Conexión en tiempo real (Socket.IO) para actualización de alertas sin recarga de página.
- Indicadores clave implementados: número de auditorías en curso, hallazgos por nivel de riesgo, tasa de detección del mes, entidades con mayor índice de riesgo fiscal, avance de plazos.

Criterio de aceptación: El dashboard carga completamente en menos de 3 segundos bajo condición de 200 usuarios concurrentes, y los datos se actualizan con latencia máxima de 5 minutos.

3.5 Módulo de Alertas y Notificaciones (RF-07)

- Trigger Engine: Servicio que escucha eventos del sistema vía Kafka (hallazgos de alto riesgo, vencimientos de plazos, umbrales de inconsistencia).
- Notification Service: Envío de notificaciones en plataforma (WebSocket push) y correo institucional vía SMTP. Tiempo máximo de envío: 60 segundos desde la detección del evento.
- Plantillas de notificación: Configurables por el Administrador del Sistema para cada tipo de evento.

3.6 Módulo de Repositorio de Auditorías (RF-08)

- Motor de búsqueda: Elasticsearch sobre PostgreSQL para búsqueda full-text con filtros por entidad, período, auditor y tipo de hallazgo.
- Historial de versiones: Cada cambio sobre un hallazgo registra la versión original, la modificación, el autor y la fecha (requerimiento HU-03).
- Control de acceso granular: Los resultados de búsqueda respetan los permisos del rol activo (RF-06).

Criterio de aceptación: Resultados de búsqueda en menos de 2 segundos sobre una base de datos de 100.000 registros.

3.7 Módulo de Gestión de Usuarios y Roles (RF-06)

- Integración LDAP/AD: Autenticación delegada al Active Directory institucional de la CGN mediante Spring Security LDAP o python-ldap3.
-

-
- Control de Acceso Basado en Roles (RBAC): Cinco roles definidos: Auditor Junior, Auditor Senior, Coordinador de Auditoría, Director y Administrador del Sistema.
 - Administración de cuentas: CRUD completo de usuarios con sincronización bidireccional con Active Directory (HU-06).
 - Sesiones y tokens: JWT con expiración configurable y refresco seguro. Autenticación completada en menos de 3 segundos.

4. DIGITALIZACIÓN DEL CÓDIGO FUENTE

Esta sección establece los estándares, convenciones y herramientas mediante los cuales el código fuente del SIAIA-CGN es producido, versionado, revisado y custodiado durante la fase de desarrollo.

4.1 Repositorio de Código Fuente

Todo el código fuente del proyecto se gestiona en un repositorio Git centralizado alojado en la infraestructura soberana de la CGN (GitLab On-Premise). No se permite el uso de repositorios en la nube pública para ningún componente del sistema.

4.1.1 Estructura del repositorio

La organización del repositorio sigue la convención monorepo con la siguiente estructura de directorios:

Directorio / Archivo	Descripción
/siaia-cgn/	Raíz del monorepo del proyecto
/frontend/	Código fuente React.js del cliente web
/backend/	Microservicios Python (FastAPI)
/api-gateway/	Punto de entrada único para todas las peticiones
/auth-service/	Autenticación LDAP y gestión de tokens JWT
/ingesta-service/	Carga y preprocesamiento de documentos
/ai-engine/	Motor de IA: modelos, pipelines y API de inferencia
/report-service/	Generación de reportes .docx y .pdf
/notification-service/	Alertas y notificaciones SMTP / WebSocket
/repository-service/	Repositorio de auditorías y búsqueda
/infrastructure/	Configuraciones Docker, Kubernetes, Terraform
/ml-models/	Notebooks de entrenamiento, modelos versionados (MLflow)
/tests/	Pruebas unitarias, de integración y de carga
/docs/	Documentación técnica, arquitectura y APIs (Swagger/OpenAPI)
/.gitlab-ci.yml	Pipeline de CI/CD del proyecto
/README.md	Guía de inicio rápido para el equipo de desarrollo

4.2 Estrategia de Branching (Gitflow Adaptado)

El equipo de desarrollo sigue la estrategia Gitflow adaptada a las restricciones del proyecto:

Rama	Propósito	Política de merge
main	Versión en producción	Solo desde release. Requiere aprobación del Líder de TI - CGN.
develop	Integración continua de features	Merge automático desde feature tras revisión de pares.
feature/XXXX	Desarrollo de una funcionalidad (referenciada al ID de Jira)	Pull Request con revisión de al menos 2 ingenieros.
release/vX.X	Preparación de versión estable	Solo corrección de bugs. Aprobado por QA antes de merge a main.
hotfix/XXXX	Corrección urgente en producción	Revisión expedita. Merge a main y develop simultáneamente.

4.3 Estándares de Codificación

Todo el código entregado al repositorio debe cumplir los siguientes estándares antes de ser mergeado a develop:

4.3.1 Backend Python

- Estilo de código: PEP 8 + Black formatter. Verificado automáticamente por pre-commit hook.
- Tipado estático: Type hints obligatorios en todas las funciones públicas. Validado con mypy.
- Linter: flake8 + pylint con configuración de umbral de puntuación mínima 8.0/10.
- Docstrings: Obligatorios en todas las clases y funciones públicas (formato Google Docstrings).
- Longitud máxima de línea: 100 caracteres.

4.3.2 Frontend TypeScript / React

- Estilo de código: ESLint + Prettier. Configuración base airbnb-typescript.
- Componentes: Functional components con hooks. Prohibido el uso de class components.
- Accesibilidad: Cumplimiento WCAG 2.1 nivel AA en todos los componentes de interfaz.
- Gestión de estado: Redux Toolkit para estado global. React Query para estado del servidor.
- Pruebas: Jest + React Testing Library. Cobertura mínima del 80% por componente.

4.3.3 Seguridad en el código

- SAST (Static Application Security Testing): Integrado en el pipeline CI con Bandit (Python) y Semgrep.
- Secrets management: Prohibido incluir credenciales, tokens o contraseñas en el código fuente. Se utiliza HashiCorp Vault integrado con K8s.
- Dependencias: Todas las librerías de terceros son escaneadas con Safety (Python) y npm audit (Node.js) antes de incorporarse.
- OWASP Top 10: Cada módulo es revisado contra la lista OWASP Top 10 antes de aprobarse para release.

4.4 Pipeline de Integración y Entrega Continua (CI/CD)

El pipeline de CI/CD se ejecuta automáticamente en GitLab CI ante cada Push a develop o Pull Request. Las etapas son:

Etapas	Nombre	Actividades	Condición de fallo
1	Lint & Format	Verificación PEP8, ESLint, Prettier, Black	Cualquier violación de estilo bloquea el pipeline.
2	Unit Tests	Ejecución de suite de pruebas unitarias	Cobertura < 80% o prueba fallida bloquea el pipeline.
3	SAST Security	Bandit, Semgrep, npm audit, Safety	Vulnerabilidad crítica o alta bloquea el pipeline.
4	Build	Construcción de imágenes Docker por microservicio	Error de compilación bloquea el pipeline.
5	Integration Tests	Pruebas de integración contra entorno de staging	Fallo en integración bloquea el merge a develop.
6	Deploy Staging	Despliegue automático en entorno de staging K8s	Solo para rama develop. Manual para release.
7	Performance Tests	k6 load test: 200 usuarios, tiempo respuesta < 3s	Degradación de rendimiento genera alerta al Líder Técnico.

5. INTEGRACIÓN DE APLICATIVOS

Esta sección describe en detalle cómo el SIAIA-CGN se conecta e interopera con los sistemas externos e internos que conforman el ecosistema tecnológico de la Contraloría General de la Nación.

5.1 Mapa de Integraciones del Sistema

El SIAIA-CGN integra los siguientes aplicativos:

Sistema	Tipo	Protocolo	Dirección	RF asociado
SIIF Nación	Externo - Ministerio de Hacienda	API REST + OAuth 2.0	Entrada (consumo)	RF-05
Active Directory CGN	Interno - Directorio corporativo	LDAP / LDAPS	Entrada (autenticación)	RF-06
Servidor SMTP Institucional	Interno - Correo CGN	SMTP + TLS 1.3	Salida (notificaciones)	RF-07
MinIO (S3 On-premise)	Interno - Almacenamiento	S3 API REST	Bidireccional	RF-01, RF-03, RF-08
Sistema de Firma Digital	Interno - PKI CGN	REST + certificado X.509	Bidireccional	RF-03

5.2 Integración con SIIF Nación (RF-05)

El SIIF Nación es el sistema financiero del Gobierno colombiano administrado por el Ministerio de Hacienda y Crédito Público. La integración permite al SIAIA-CGN consumir en tiempo real las ejecuciones presupuestales, órdenes de pago y contratos activos.

5.2.1 Especificación técnica de la integración

- Tipo de integración: Consumo de API REST expuesta por SIIF Nación bajo el modelo de servicios compartidos del Estado colombiano.
 - Autenticación: OAuth 2.0 con flujo Client Credentials. Las credenciales de cliente son provistas por el Ministerio de Hacienda y custodiadas en HashiCorp Vault.
 - Endpoints principales consumidos: /ejecucion-presupuestal, /ordenes-pago, /contratos-activos, /cdps (Certificados de Disponibilidad Presupuestal).
 - Sincronización: Programada cada 5 minutos mediante un scheduler (APScheduler) con reintentos automáticos en caso de fallo (máximo 3 reintentos con backoff exponencial).
 - Disponibilidad requerida: Sincronización exitosa en el 99.5% de las solicitudes. Tiempo de respuesta menor a 5 segundos (RF-05).
-

-
- Manejo de fallos: Ante indisponibilidad del SIIF, el sistema continúa operando con los datos de la última sincronización exitosa, notificando al Administrador la brecha de datos.

5.2.2 Flujo de datos SIIF → SIAIA-CGN

1. El scheduler dispara la solicitud de sincronización con token OAuth 2.0 vigente.
2. El Integración Service consulta los endpoints definidos del SIIF Nación.
3. La respuesta JSON es validada contra el esquema definido (JSON Schema validation).
4. Los datos son transformados al modelo de datos interno del SIAIA-CGN (ETL).
5. Se persisten en PostgreSQL y se publican en el topic Kafka 'siif.datos.nuevos'.
6. El Motor de IA suscrito al topic inicia el análisis automático de los nuevos datos.
7. El resultado queda disponible en el dashboard y en la bandeja de hallazgos.

5.3 Integración con Active Directory / LDAP (RF-06)

La autenticación de usuarios del SIAIA-CGN se delega completamente al Active Directory institucional de la CGN. El sistema no almacena contraseñas de usuarios.

- Protocolo: LDAPS (LDAP sobre TLS 1.3) para garantizar la confidencialidad de las credenciales en tránsito.
- Flujo de autenticación: El usuario ingresa credenciales institucionales → SIAIA-CGN consulta AD vía LDAP → Si válido, genera JWT con claims de rol → JWT firmado se retorna al cliente.
- Sincronización de roles: Los grupos del AD se mapean a los roles del sistema. La asignación se gestiona desde el módulo de Administración con sincronización automática diaria.
- Tiempo de autenticación: Menor a 3 segundos desde el envío de credenciales hasta la sesión activa.
- Bloqueo de sesión: Sesión inactiva por más de 30 minutos se invalida automáticamente (política de seguridad institucional).

5.4 Integración con el Servidor SMTP Institucional (RF-07)

- Protocolo: SMTP sobre TLS 1.3 (STARTTLS), puerto 587.
 - Credenciales: Cuenta de servicio institucional dedicada al sistema (siaia-notificaciones@contraloria.gov.co), gestionada en HashiCorp Vault.
 - Plantillas de correo: HTML responsivo con identidad visual de la CGN, configurables por el Administrador.
 - Tiempo máximo de entrega: El Notification Service garantiza el envío al servidor SMTP en menos de 60 segundos desde la detección del evento disparador.
 - Reintento y logs: En caso de fallo SMTP, hasta 3 reintentos con backoff. Todos los envíos quedan registrados en el log inmutable del sistema.
-

5.5 Integración con MinIO – Almacenamiento de Objetos (RF-01, RF-03, RF-08)

- Despliegue: Clúster MinIO On-Premise en infraestructura soberana de la CGN con redundancia RAID-6.
- Buckets definidos: siaia-documentos-brutos (documentos originales cargados), siaia-documentos-procesados (datos extraídos y estructurados), siaia-reportes (reportes .docx y .pdf finales), siaia-evidencias (archivos de evidencia asociados a hallazgos).
- Cifrado: AES-256 para datos en reposo sobre todos los buckets. TLS 1.3 para transferencia.
- Control de acceso: Políticas de bucket por servicio (least privilege). El Frontend nunca accede directamente a MinIO; lo hace siempre a través del backend.
- Retención: Los documentos se retienen por el período legal establecido por la CGN (mínimo 5 años), con política de lifecycle configurada en MinIO.

5.6 Integración con el Sistema de Firma Digital PKI – CGN (RF-03, HU-05)

- Estándar: Firma digital basada en certificados X.509 emitidos por la PKI institucional de la CGN.
 - Flujo de firma: El Director genera el reporte → Solicita firma digital desde el módulo de reportes → El sistema invoca la API PKI institucional → Se incrusta la firma en el PDF → El documento firmado se almacena con sello de tiempo RFC 3161.
 - Verificación de integridad: Hash SHA-256 del documento almacenado permite verificar que el reporte no ha sido alterado post-firma (HU-05).
-

6. PLAN DE PRUEBAS DURANTE EL DESARROLLO

Las pruebas durante la fase de desarrollo siguen el enfoque shift-left: cada módulo es probado de forma continua durante su construcción, sin esperar a la fase formal de UAT (F5). Esto garantiza la detección temprana de defectos y el cumplimiento de la cobertura mínima del 80% (RNF-07).

6.1 Tipos de Pruebas en Desarrollo

Tipo	Descripción	Herramienta	Responsable
Pruebas Unitarias	Verifican el comportamiento de funciones y clases de forma aislada usando mocks para dependencias externas.	pytest (Python) / Jest (React)	Desarrollador
Pruebas de Integración	Verifican la comunicación entre microservicios y con bases de datos reales en entorno staging.	pytest + Docker Compose	Desarrollador + QA
Pruebas de Contrato API	Verifican que las APIs cumplen el contrato OpenAPI definido, incluyendo la API SIIF.	Dredd / Pact	QA
Pruebas de Seguridad (SAST)	Análisis estático del código para detectar vulnerabilidades antes del merge.	Bandit, Semgrep, npm audit	Pipeline CI/CD
Pruebas de Rendimiento	Simulan carga de 200 usuarios concurrentes para verificar tiempos de respuesta.	k6 / Locust	QA + DevOps
Pruebas de Precisión IA	Validan métricas de Precisión y Recall del modelo sobre el dataset histórico CGN.	pytest + scikit-learn metrics	Científico de Datos

6.2 Criterios de Calidad para Avance de Fase

Un módulo puede declararse listo para UAT (Fase F5) solo cuando cumple la totalidad de los siguientes criterios:

- Cobertura de pruebas unitarias: igual o superior al 80% de líneas de código (RNF-07).
 - Cero vulnerabilidades críticas o altas pendientes de corrección en el SAST.
 - Todos los criterios de aceptación técnica del módulo cumplidos (ver Sección 3).
 - Pipeline CI/CD completado exitosamente en las últimas 3 ejecuciones consecutivas.
 - Documentación de la API del módulo actualizada en el portal Swagger/OpenAPI interno.
 - Revisión de pares (code review) aprobada por al menos dos ingenieros del equipo.
-

7. EQUIPO DE DESARROLLO Y RESPONSABILIDADES

Rol	Perfil requerido	Responsabilidades principales
Líder de Desarrollo	Ingeniero Senior 5+ años	Arquitectura técnica, revisión de código crítico, toma de decisiones técnicas, reporte al Comité directivo.
Científico de Datos / ML Engineer	PhD o MSc Ciencias de Datos, 3+ años ML	Diseño, entrenamiento y evaluación del motor de IA. Versionamiento de modelos con MLflow.
Ingeniero Backend (x2)	Desarrollador Python Senior	Implementación de microservicios FastAPI, integración SIIF, gestión de datos PostgreSQL/Kafka.
Ingeniero Frontend	Desarrollador React/TypeScript Senior	Implementación del cliente web, dashboard, formularios y visualizaciones.
Ingeniero DevOps / Infraestructura	Especialista K8s + GitLab CI	Pipeline CI/CD, configuración de entornos, monitoreo, respaldos.
Analista QA	Ingeniero de Calidad 3+ años	Plan de pruebas, ejecución de pruebas de integración y rendimiento, gestión de defectos en Jira.
Analista de Seguridad	Especialista en Ciberseguridad	Revisión de cumplimiento OWASP, pruebas de penetración básicas, gestión de secretos y PKI.
Coordinador TI - CGN	Líder técnico institucional	Punto de contacto para acceso a AD, SIIF, SMTP y PKI. Validación de requerimientos técnicos.

8. CRONOGRAMA DETALLADO DE DESARROLLO

El cronograma de la Fase de Desarrollo (F2 – F3) corresponde a los meses 2 al 5 del proyecto, distribuidos en sprints de dos semanas bajo metodología Scrum adaptada a las restricciones del contrato.

Sprint	Semanas	Actividades principales	Entregables	Módulos
S1	7 – 8	Configuración entornos Dev/Staging. Setup repositorio GitLab, pipeline CI/CD base. Configuración BD y Kafka.	Entornos operativos. Pipeline CI activo.	Infraestructura
S2	9 – 10	Módulo de Autenticación LDAP/AD. API Gateway. Módulo de Gestión de Usuarios y Roles (CRUD + AD sync).	Auth Service v1.0. User Service v1.0.	RF-06
S3	11 – 12	Módulo de Ingesta: File Upload Service, Document Preprocessor (OCR + parseo), Data Extractor Pipeline.	Ingesta Service v1.0. Pruebas unitarias.	RF-01
S4	13 – 14	Entrenamiento y evaluación del Modelo de IA. Implementación del Rule Engine. Métricas Precision/Recall.	Modelo IA v1.0 con métricas documentadas.	RF-02
S5	15 – 16	API de inferencia del Motor IA. Integración Ingesta → Motor IA vía Kafka. Pruebas integración E2E.	AI Engine API v1.0. Integración activa.	RF-02
S6	17 – 18	Dashboard frontend (React). BFF backend. Indicadores clave, gráficos de tendencias. WebSockets alertas.	Dashboard v1.0 funcional.	RF-04
S7	19 – 20	Integración API SIIF Nación (sincronización, ETL, manejo de fallos). Pruebas de contrato API.	SIIF Integration v1.0. Sincronización activa.	RF-05
S8	21 – 22	Módulo de Reportes (Template Engine, PDF Renderer, Digital Signature). Repositorio de auditorías (búsqueda).	Report Service v1.0. Repository Service v1.0.	RF-03, RF-08
S9	23 – 24	Módulo de Alertas y Notificaciones SMTP. Pruebas de rendimiento (200 usuarios). Corrección de hallazgos.	Notification Service v1.0. Informe de rendimiento.	RF-07

S10	25 – 26	Integración final de todos los módulos. Pruebas de sistema completo. Documentación técnica final. Entrega a UAT.	Sistema integrado v1.0 listo para UAT. Docs.	Todos los RF
------------	---------	--	--	--------------

9. GESTIÓN DE RIESGOS DE DESARROLLO

ID	Riesgo	Prob.	Impacto	Plan de mitigación
R01	Retraso en entrega de datos históricos de la CGN para entrenamiento del modelo de IA.	Alta	Alto	Iniciar con datos sintéticos basados en patrones definidos por auditores expertos. Definir fecha límite de entrega de datos reales: semana 8.
R02	Indisponibilidad o cambios en la API del SIIF Nación.	Media	Alto	Desarrollar una capa de abstracción (adapter) y mock del SIIF para desarrollo. Formalizar SLA con Ministerio de Hacienda desde el mes 1.
R03	Precisión del modelo de IA por debajo del 85% exigido.	Media	Alto	Incluir ciclos de ajuste de hiperparámetros en el sprint S4. Contar con expertos auditores para etiquetado de datos de refuerzo.
R04	Vulnerabilidades de seguridad detectadas tardíamente.	Baja	Crítico	SAST integrado en pipeline desde el sprint S1. Revisión de seguridad formal al final de S5 y S9.
R05	Rotación de personal clave del equipo contratista.	Baja	Alto	Documentación técnica continua y detallada. Knowledge transfer sessions semanales. Pair programming en módulos críticos.

10. ENTREGABLES DE FASE Y CRITERIOS DE CIERRE

10.1 Entregables formales al cierre de la Fase F2-F3

- Código fuente completo versionado en GitLab On-Premise de la CGN, con historial de commits limpio.
- Imágenes Docker de todos los microservicios publicadas en el Registry privado de la CGN.
- Documentación de APIs en portal Swagger/OpenAPI interno (100% de endpoints documentados).
- Informe de cobertura de pruebas unitarias con cobertura mínima del 80% por módulo.
- Informe de pruebas de rendimiento bajo carga de 200 usuarios concurrentes.
- Informe de análisis de seguridad SAST sin vulnerabilidades críticas o altas abiertas.
- Manual de instalación y despliegue del sistema en Kubernetes.

-
- Modelo de IA v1.0 entrenado con métricas documentadas (Precision $\geq$ 85%, Recall $\geq$ 80%).
 - Acta de entrega y aprobación de Fase F2-F3 firmada por el Líder de Proyecto y el Director de TI - CGN.

10.2 Criterio de cierre de la Fase de Desarrollo

La Fase de Desarrollo se declara oficialmente cerrada cuando el Comité Directivo recibe y aprueba todos los entregables listados en la sección 10.1, y el sistema es desplegado exitosamente en el entorno de staging listo para iniciar la Fase F5 de Pruebas UAT y Capacitación.